

Mizan.ai — Build Handoff

Feature E: ZATCA/VAT Calculator & Validator

Feature C: Document Comparator — Monthly Reconciliation

This document consolidates the full design discussion for both features into a single build-ready reference. It is intended for the coding agent (Claude Code) implementing these features in **agents/zatca_calculator/** and **agents/doc_comparator/**.

Global Build Instructions — Apply to Both Features

1. Two separate features, two separate pages

The calculator/validator and the comparator/reconciliation tool are **two distinct features on Mizan**, each with its **own page in the UI** — following the same pattern already established for the Translator, Format Converter, and RAG chat pages. Do not combine them into one page or one workflow. They may share backend logic (see reuse notes below) but are presented and navigated to separately.

2. Prefer LangGraph / long-term-memory course patterns

Wherever applicable, use the tools, libraries, and patterns covered in DeepLearning.AI's LangGraph and long-term agentic memory course, in preference to ad hoc alternatives. This includes LangGraph's StateGraph orchestration pattern, its checkpoint/store abstractions for persistence, and its conventions for tool-calling nodes, human-in-the-loop interrupts, and state management.

3. Code locations

Two empty directories have already been created for this work — write all new code there:

- **agents/zatca_calculator/** — Feature E (calculator + validator)
- **agents/doc_comparator/** — Feature C (comparator / monthly reconciliation)

4. Reuse existing code wherever applicable

Do not rewrite logic that already exists in the **agents/** directory. Specifically:

- **persistence.py** — `store_customer_memory()` and `get_customer_memories()` are already implemented and are exactly what the calculator's customer-history feature needs. Reuse directly, do not reimplement.
- **langgraph_chatbot.py** — use its overall LangGraph structure (StateGraph node/edge pattern, the Postgres-backed checkpoint/store setup with in-memory fallback via PostgresSaver/PostgresStore and MemorySaver/InMemoryStore) as the structural template for how both new agents should be wired up.
- **modal_app.py / model_loader.py** — reuse the already-deployed Modal model endpoints (QwenBrain /generate with confirmed working tool-calling, Translator /translate, Embedder /embed) rather than deploying new model infrastructure for either feature.

Feature E — ZATCA / VAT Calculator & Validator

Positioning & Key Findings

This feature is really **one shared calculation engine exposed through two tools**: `calculate_vat` (compute VAT from scratch on a document) and `validate_zatca_form` (recompute independently, then compare against whatever numbers are already printed on the document).

Real target audience: businesses and individuals *without* SAP/Odoo or any proper accounting system — freelancers, small vendors who cannot afford or have not yet set up real accounting software. Businesses already running a compliant ERP have this problem largely solved already: SAP/Odoo compute VAT automatically from pre-configured tax rules (product tax settings, fiscal positions) decided once in advance — they never have to infer tax treatment from a document. Mizan has no such setup, so it must infer tax treatment fresh from each document, every time.

Important legal context confirmed during research: since 4 December 2021, e-invoicing (ZATCA Phase 1) has been mandatory for **all** VAT-registered businesses in Saudi Arabia — handwritten or plain PDF invoices are no longer technically compliant. This means a properly compliant business's own invoices already have VAT correctly calculated before Mizan ever sees them. The narrower, more clearly validated need is **checking/validating** an existing document, not generating a calculation from a blank slate. The calculate-from-scratch scenario mainly applies to businesses in the gap before they're properly set up, or to quotes/estimates.

Highest-risk inference in the whole pipeline: tax category classification per line item (standard / zero-rated / exempt). This is the one thing Odoo/SAP never have to guess (it's pre-configured) and Mizan always does. This should be the field most visibly flagged for review.

Explicitly rejected for v1, after being pressure-tested: a "should I register for VAT voluntarily" decision helper. No evidence of active demand was found (people who think about this ahead of time appear to default to asking a human tax consultant, not a calculator), and it's a one-time decision with no recurring engagement, unlike the calculator/validator. Keep as backlog only — do not build in v1.

Input Formats & Detection Order

Documents should be checked in this order, fastest/most reliable first:

#	Format	Handling
1	ZATCA-compliant XML / UBL 2.1 file	Parse directly as structured XML — no OCR needed.
2	PDF/A-3 with embedded XML	Check for an embedded XML attachment first; extract that instead of OCR-ing the visual layout.
3	Image or PDF with a visible QR code	Decode the QR (TLV, Base64-encoded) first for 5 reliable fields instantly: seller name, VAT number, etc.
4	Everything else (scanned invoice, photo, etc.)	This is the primary PDF/image behavior. Goes through the existing Docling-based segment.

Docling already auto-detects file type internally (PDF, images incl. JPG/PNG, DOCX, PPTX, XLSX) and handles each appropriately — OCR via an embedded VLM + PaddleOCR for images (PaddleOCR specifically handles Arabic well), native text/table extraction for structured formats. Do not build a separate extraction dispatcher for file types — this already happens inside Docling.

Standard vs Simplified invoice: Simplified (B2C) invoices will not have a buyer VAT number. Extraction should treat a missing buyer TRN on these as expected-normal, not a low-confidence/missing-field flag.

Pipeline Steps

Step 1 — Input Router (automatic, NOT an MCP tool)

Detects format and routes to the correct extraction path per the table above. Kept as plain automatic code, not an agent-callable tool — this is a mechanical fact-check about the file, not a judgment call, and letting an agent skip or mis-order it would break everything downstream.

Step 2 — Extraction & Classification (automatic, NOT an MCP tool)

Docling performs raw extraction. QwenBrain (via the existing /generate endpoint) performs tax-category classification per line item — the one genuinely ambiguous judgment call in the pipeline, since it requires understanding what an item actually is, not just reading digits off the page. Each extracted field gets a confidence score.

Language handling: QwenBrain should extract **directly into a fixed, English-labeled schema** (description, amount, tax_category, etc.) even when the source document is in Arabic — this mapping is part of the extraction step itself, not a separate translation stage. The calculation engine (Step 4) never receives or needs the original language text at

all — it only ever operates on numbers and category labels. Do not insert a translate-then-extract step; it adds a needless source of error to a step that doesn't require it.

Step 3 — Human-in-the-Loop Review Module

One single review screen with two parts, both fully editable:

- **Setup confirmation section (top):** system's best guess for — company vs. individual (affects identity field: CR number vs. national ID/Iqama), local vs. foreign counterparty (affects whether tax applies at all), B2B vs. B2C, and related-party/intra-group (usually 0% between related companies). Person can override any of these.
- **Line-item table (below):** extracted description / amount / tax category per line. Low-confidence fields are visually flagged. Every field is editable inline.
- **One button: "Confirm & Calculate."** No separate review-then-approve steps — nothing is calculated until this single confirmation, and whatever is on screen at that point (edited or not) is exactly what gets calculated.

Step 4 — Core Tax Calculation Engine (pure deterministic Python — no LLM)

This is the shared heart of both tools. Rules:

- Three tax categories: **standard** (15%), **zero-rated** (0%, input VAT reclaimable), **exempt** (0%, input VAT not reclaimable). Capture the category on every line even though zero-rated and exempt compute identically today — costs nothing to store and unblocks a future input-VAT-recovery feature without rework.
- **Round per line item, not once at the end.** Each line: $\text{taxable_base} \times \text{rate} \rightarrow \text{line VAT}$, rounded to 2 decimals at the line level. This is the detail that trips up naive calculators and causes exactly the kind of mismatch the validator exists to catch.
- Subtotal = sum of line taxable bases. Total VAT = sum of the already-rounded line VAT amounts. Grand total = subtotal + total VAT.
- Multi-rate invoices (mixed standard/zero-rated lines on one invoice) are calculated per line, never on the invoice total.
- Edge cases are handled by **flagging for manual review only**, never by automatic calculation: reverse charge / import of services → flag "verify manually"; related-party or tax-unit zero-rating → flag for confirmation rather than assume.
- `taxpayer_type` (company vs. individual) is a single shared field, not a separate calculation path. It only changes: (1) which identity field is expected, (2) whether the non-resident digital-service registration rule applies, (3) which identifier populates the ZATCA XML seller-ID field. The math itself is identical either way.

Step 5 — Two thin tool wrappers (MCP tools) around the engine

- `calculate_vat` — calls the engine, returns the computed numbers. Nothing else.
- `validate_zatca_form` — calls the engine, then compares its output against whatever numbers are already printed on the document, returning match/mismatch with both values shown side by side (e.g. "document shows SAR 142.50, recalculated value is SAR 150.00"). The engine itself never looks at the printed numbers — it only ever computes fresh — so this comparison is genuine, not circular.

Step 6 — Structural Rules Module (separate from the math; used only by the validator)

Required-field checks, VAT number format validation, date sanity checks, cross-field consistency (buyer ≠ seller, no future dates, registration-number checksum where applicable). Kept as its own module, separate from the calculation engine, so it can eventually reference Feature A's RAG knowledge base to cite the actual regulation behind a failure — without touching the calculation engine at all.

Step 7 — Report Generation (*generate_report* tool)

Existing tool (ReportLab for PDF, python-docx for DOCX, openpyxl for XLSX), extended with a **language** parameter: **Arabic or English**. Numbers never change by language — only item descriptions/labels are translated when English output is requested from an Arabic-source document, reusing the already-deployed MADLAD-400 translator. Bilingual label templates are needed for standard report fields (e.g. "Invoice Total" / "■ ■ ■ ■ ■ ■ ■ ■ ■ ■").

Step 8 — Batch / multi-document handling

Run the per-document pipeline (extract → review → calculate) across multiple files in one job — e.g. a month's worth of invoices — and combine everything into **one** review table at the end, rather than one screen per file.

Step 9 — Customer memory (*reuse, do not rebuild*)

Use the existing `store_customer_memory()` / `get_customer_memories()` functions in `persistence.py` directly. Save a short note after each job (customer name, what was processed, when).

RAG Knowledge Base — Not Required

The calculator/validator functions completely without Feature A's RAG knowledge base. VAT rates are fixed/hardcoded; structural checks are fixed rules. The only optional future use is having the validator cite the actual regulation text behind a mismatch — a nice-to-have polish item for later, not a dependency for v1.

MCP Architecture — Final Decision

One MCP server, three tools: `calculate_vat`, `validate_zatca_form`, `generate_report`.

Rule of thumb applied when deciding what is a tool vs. what stays automatic code:

- If skipping a step or getting its order wrong could break the pipeline → keep it **automatic** (applies to: Input Router, Extraction, the Human-in-the-Loop review screen).
- If it's a genuine "should I do this now, and how" decision tied to what the user asked for → make it an **MCP tool** (applies to: `calculate_vat`, `validate_zatca_form`, `generate_report`).

QwenBrain (already deployed on Modal, tool-calling confirmed working end-to-end per the deployment lessons-learned doc) is the orchestrating agent / MCP client that discovers and calls these three tools.

Explicitly Out of Scope for v1

- "Should I register for VAT voluntarily" decision tool — backlog only.
- Any live/real-time connection to a user's accounting system — Mizan only ever works from uploaded files, never a live database connection.

Feature C — Document Comparator: Monthly Reconciliation

Positioning & Key Findings

Originally scoped generically ("any two documents with something in common"). Refined through discussion into a specific flagship use case: **monthly reconciliation of an SAP/Odoo export against a second document** (e.g. a vendor statement, a bank statement).

This is the strongest validated "genuine add-on to an existing ERP" use case in the whole project. SAP/Odoo structurally *cannot* do this reconciliation themselves — it requires comparing their own data against an independently generated document from an outside party (the vendor's own system, or a bank), and that comparison has to happen outside both systems by definition.

Honest competitive note: this is **not** an empty market the way the individual VAT-registration question was. Dedicated vendor/supplier statement reconciliation software already exists globally (Xelix, Medius, Trintech, HighRadius, and others), and industry data shows 1 in 3 supplier statements contain errors, with real financial upside from fixing this (one case study saw a 96% reduction in duplicate payments after automating). All of these existing tools are Western/enterprise-focused — none are Arabic-native or scaled/priced for the Saudi SME segment. Mizan's angle is Arabic-first, Saudi-SME scaled, and reuses plumbing already built for the calculator — not "nobody else does this."

What currently happens without Mizan: at month-end, the user exports data from SAP/Odoo, then **manually builds an entirely new worksheet by hand** comparing it against the second document — line by line, matching items, flagging what doesn't line up, explaining differences (timing, disputes, missing invoices). There is no pre-existing form or template for this — it is built from scratch, every month, and this manual process is where errors creep in. Mizan's job is to build that same third worksheet automatically instead of by hand.

v1 scope limit, explicitly confirmed: month-end batch activity only — not real-time or continuous reconciliation. Widening to other cadences is a later phase, not part of this build.

What gets compared against the SAP/Odoo export is **generic in v1**, not fixed to one specific document-pair type. The tool should work on "SAP/Odoo export + whatever second document the user provides," rather than being hardcoded to one specific counterpart (e.g. only vendor statements).

Input Formats

Side	Typical format	Handling
SAP/Odoo export	Clean, structured — Excel (.xlsx) or a well-formatted PDF table, already organized in rows/columns	One if not tied to a table, already organized in rows/columns for T/B, before
Second document (e.g. vendor statement)		If it also comes from a proper accounting system: equally clean (Excel/PDF)

Pipeline / Flow — Final Design

This flow was explicitly refined during design: the review step should **not** happen right after raw extraction (extraction from a clean spreadsheet is basically always correct and doesn't need double-checking). The review step belongs **after matching**, focused only on genuinely unclear cases.

Step 1 — Read both files

Structured parse for clean Excel/PDF exports; Docling extraction (reusing the calculator's pipeline) for messy documents.

Step 2 — Automatic quiet matching pass

Match items between the two documents wherever they line up cleanly (matching reference number, matching amount). **No review shown for these** — they are simply resolved and set aside.

Step 3 — Fallback matching

For anything that didn't match cleanly by reference number, fall back to matching by amount and date proximity.

Step 4 — Human-in-the-Loop review screen

Shows **only** the leftover unclear or unmatched items — never the clean matches. For each flagged item, the person can confirm it's actually a match, dismiss it, or add an explanation (e.g. a timing difference, a dispute). This is deliberately the single review moment in the whole flow.

Step 5 — Generate the final reconciliation file

Produces the same output a person would have built by hand — a matched summary plus explained/confirmed exceptions — as a downloadable file (reusing the same report-generation approach as the calculator, PDF/DOCX/XLSX).

This exact pattern — auto-match quietly, surface only exceptions, ask the human to confirm just the unclear ones — was cross-checked against how real reconciliation software in the market already works (Xeliox, Medius, and others all converge on "capture statement, extract, match against records automatically, flag exceptions only"). This confirms it as the right design pattern, not just a convenient shortcut for Mizan.

Explicitly Left Open / Not Fully Detailed for v1

- Exact fuzzy-matching thresholds for the amount+date-proximity fallback — the approach is agreed in principle (reference number first, amount+date proximity as fallback) but not detailed to an implementation-ready threshold/algorithm level. Use reasonable judgment here or confirm with Mohammed before finalizing.
- No specific document-pair type is hardcoded (e.g. vendor statement vs. bank statement) — keep the second input generic in v1.

Appendix — Quick Reference

Feature E MCP Tools

Tool	Purpose
calculate_vat	Compute VAT from extracted/confirmed line items. Returns computed numbers only.
validate_zatca_form	Recompute independently via the same engine, then compare against the numbers already printed on the o
generate_report	Produce the final output file (PDF/DOCX/XLSX) in Arabic or English.

Shared Design Principles Across Both Features

- Human-in-the-loop is mandatory before any final output is produced — but placed at the point of genuine ambiguity, not as a blanket "review everything" step.
- One confirmation action, not a multi-step review-then-approve flow.
- Show/flag only what needs attention; hide what's already resolved or clean.
- Reuse existing plumbing (extraction, persistence, deployed models) rather than duplicating logic between the two features.
- Keep automatic, pipeline-critical steps as plain code; expose only genuine agent decisions as MCP tools.

End of handoff document. Prepared from the full Mizan.ai design discussion covering both features.